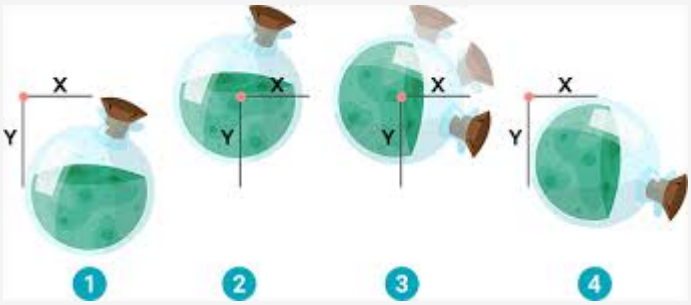
Transformations avec Canvas

Les transformations avec Canvas permettent de modifier la position, la taille et l'orientation des éléments dessinés sur le canevas. Dans les jeux vidéo, elles servent à faire tourner des avions, agrandir des power-ups qui pulsent, ou faire pivoter des personnages selon leur direction. Nous aurons principalement besoin de trois transformations : la translation (déplacement), la rotation et l'échelle (redimensionnement) ainsi que des fonctions pour sauvegarder et restaurer l'état du canevas (`save()` et `restore()`).

Pourquoi save() et restore() ?



Le problème : Sans save()/restore(), toutes les transformations s'accumulent et affectent TOUT ce qui est dessiné après.

```
// MAUVAIS EXEMPLE - Sans save/restore
oContexte.translate(100, 100);
oContexte.rotate(0.5);
oContexte.fillRect(0, 0, 50, 50); // Ce carré tourne ☑

oContexte.fillRect(200, 200, 30, 30); // Ce carré tourne AUSSI ! ✖
```

La solution : `save()` sauvegarde l'état actuel, `restore()` le remet comme avant.

```
// BON EXEMPLE - Avec save/restore
oContexte.save(); // Sauvegarder l'état normal
oContexte.translate(100, 100);
oContexte.rotate(0.5);
oContexte.fillRect(0, 0, 50, 50); // Ce carré tourne ☑
oContexte.restore(); // Remettre l'état normal

oContexte.fillRect(200, 200, 30, 30); // Ce carré ne tourne pas ☑
```

Définir le point d'origine

Par défaut, le point d'origine (0, 0) est situé en haut à gauche du canevas. Pour définir un autre point d'origine, on utilise la méthode `translate(x, y)`, où `x` et `y` sont les coordonnées du nouveau point d'origine. Vous devez appeler cette méthode avant de dessiner l'élément que vous voulez déplacer et avant d'appliquer d'autres transformations comme la rotation ou l'échelle.

Rotation

Pour faire une rotation, on utilise la méthode `rotate(angle)`, où `angle` est en radians. 2 radians équivalent à 360 degrés.

Pour convertir des degrés en radians, on peut utiliser la formule suivante : `radians = degres * (Math.PI / 180)`. Pour convertir des radians en degrés, on peut utiliser la formule suivante : `degres = radians * (180 / Math.PI)`.

Voici un exemple de rotation d'un carré :

```
const oCanvasHTML = document.querySelector("canvas");
const oContexte = oCanvasHTML.getContext("2d");

let nAngle = 0;
let nPositionX = 100;
let nPositionY = 100;

function init() {
    setInterval(boucleJeu, 1000 / 60); // 60 FPS
}

function boucleJeu() {
    oContexte.clearRect(0, 0, oCanvasHTML.width, oCanvasHTML.height); // Efface le
canevas
    nAngle += 0.05; //On tourne de 0.05 radian à chaque frame (environ 2.86
degrés)

    oContexte.save(); // Sauvegarde l'état actuel du canevas pour ne faire tourner
que ce qui suit
    oContexte.translate(nPositionX, nPositionY); // Déplace le point d'origine au
centre du carré
    oContexte.rotate(nAngle); // Convertit les degrés en radians et fait la
rotation
    oContexte.fillRect(-25, -25, 50, 50); // Dessine le carré centré sur le
nouveau point d'origine
    oContexte.restore(); // Restaure l'état du canevas avant la rotation
}
```

Exemple de jeu : Avion qui tourne selon sa direction

```
let avionX = 200;
let avionY = 200;
let avionAngle = 0;
let vitesse = 2;
let avionImage = new Image();
avionImage.src = "images/avion.png"; // Assurez-vous d'avoir une image d'avion

function boucleJeu() {
    oContexte.clearRect(0, 0, oCanvasHTML.width, oCanvasHTML.height);
```

```
    // Déplacer l'avion selon son angle. L'avion va avancer dans la direction où
il pointe.
    avionX += Math.cos(avionAngle) * vitesse; //Math.cos() retourne la coordonnée
X sur le cercle afin de savoir combien avancer horizontalement
    avionY += Math.sin(avionAngle) * vitesse; //Math.sin() retourne la coordonnée
Y sur le cercle afin de savoir combien avancer verticalement

    // Dessiner l'avion qui pointe dans sa direction
    oContexte.save();
    oContexte.translate(avionX, avionY);
    oContexte.rotate(avionAngle);

    // Dessiner avion avec une image
    oContexte.drawImage(avionImage, -avionImage.width / 2, -avionImage.height /
2);

    oContexte.restore(); // Restaure l'état du canevas avant la rotation de
l'avion pour le reste du dessin
}

// Contrôles : tourner l'avion
document.addEventListener("keydown", function (evenement) {
    if (evenement.key === "ArrowLeft") {
        avionAngle -= 0.1;
    }
    if (evenement.key === "ArrowRight") {
        avionAngle += 0.1;
    }
});
```

Échelle

Pour redimensionner un élément, on utilise la méthode `scale(x, y)`, où `x` est le facteur d'échelle horizontal et `y` est le facteur d'échelle vertical. Un facteur de 1 signifie que l'élément conserve sa taille originale, un facteur supérieur à 1 agrandit l'élément, et un facteur inférieur à 1 le rétrécit. Voici un exemple de redimensionnement d'un carré :

```
const oCanvasHTML = document.querySelector("canvas");
const oContexte = oCanvasHTML.getContext("2d");

let nAgrandissement = 1;
let bGrossit = true;
let nPositionX = 100;
let nPositionY = 100;

function init() {
    setInterval(boucleJeu, 1000 / 60); // 60 FPS
}

function boucleJeu() {
    oContexte.clearRect(0, 0, oCanvasHTML.width, oCanvasHTML.height); // Efface le
canevas
```

```
    // Change la taille du carré, permet un effet de pulsation
    if (bGrossit) {
        nAgrandissement += 0.01;
        if (nAgrandissement >= 2) {
            bGrossit = false;
        }
    } else {
        nAgrandissement -= 0.01;
        if (nAgrandissement <= 1) {
            bGrossit = true;
        }
    }

    oContexte.save(); // Sauvegarde l'état actuel du canevas pour ne faire grossir
que ce qui suit
    oContexte.translate(nPositionX, nPositionY); // Déplace le point d'origine au
centre du carré
    oContexte.scale(nAgrandissement, nAgrandissement); // Applique l'échelle
    oContexte.fillRect(-25, -25, 50, 50); // Dessine le carré centré sur le
nouveau point d'origine
    oContexte.restore(); // Restaure l'état du canevas avant l'échelle
}
```

Exemple de jeu : Power-up animé en pulsation quand on s'approche

```
let joueurX = 100;
let joueurY = 100;
let powerUpX = 300;
let powerUpY = 200;
let tailleNormale = 20;

function boucleJeu() {
    oContexte.clearRect(0, 0, canvas.width, canvas.height);

    // Calculer la distance entre joueur et power-up
    let ecartX = powerUpX - joueurX;
    let ecartY = powerUpY - joueurY;
    let distance = Math.sqrt(ecartX * ecartX + ecartY * ecartY);

    // Plus on est proche, plus le power-up pulse
    let facteurPulsation = 1;
    if (distance < 100) {
        facteurPulsation = 1 + (100 - distance) / 100; // Entre 1 et 2
    }

    // Dessiner le joueur (carré bleu)
    oContexte.fillStyle = "blue";
    oContexte.fillRect(joueurX - 15, joueurY - 15, 30, 30);

    // Dessiner le power-up qui pulse
    oContexte.save();
```

```
    oContexte.translate(powerUpX, powerUpY);
    oContexte.scale(facteurPulsation, facteurPulsation);

    oContexte.fillStyle = "gold";
    oContexte.fillRect(-tailleNormale / 2, -tailleNormale / 2, tailleNormale,
tailleNormale);

    oContexte.restore();
}

// Déplacement du joueur
document.addEventListener("keydown", function (e) {
    if (e.key === "ArrowLeft") {
        joueurX -= 5;
    }
    if (e.key === "ArrowRight") {
        joueurX += 5;
    }
    if (e.key === "ArrowUp") {
        joueurY -= 5;
    }
    if (e.key === "ArrowDown") {
        joueurY += 5;
    }
});
```

Cas d'usage courants dans les jeux

Quand utiliser `rotate()` :

- **Véhicules** : Voitures, avions, vaisseaux qui tournent selon leur mouvement
- **Projectiles** : Flèches, missiles qui pointent vers leur destination
- **Hélices** : Éléments qui tournent en continu
- **Personnages** : Qui se retournent selon leur direction

Quand utiliser `scale()` :

- **Power-ups** : Qui pulsent pour attirer l'attention
- **Ennemis** : Qui grossissent quand ils accumulent des dégâts
- **Effets** : Explosions qui grandissent puis rétrécissent
- **Interface** : Boutons qui grossissent au survol

Quand utiliser `translate()` :

- **Centrer les rotations** : Pour faire tourner un objet sur lui-même
- **Caméra** : Pour suivre le joueur (translate tout le monde sauf le joueur)
- **Effets de tremblement** : Déplacer tout légèrement de manière aléatoire

```

```